INTELLIRITY

INTERNAL & CLIENT-FACING REFERENCE

Security operations for AI.

The complete guide to what Intellirity does, how it works, what each module solves, how clients buy and use it, what is live today, and what is still deliberately unfinished.

INTELLIRITY

Document version 1.0 · 7 November 2026

Product: https://www.intellirity.com

Scope: marketing site, free scanner, console, 16 engines, 9 paid modules, billing, security model, operations, limitations

Use this document to answer client, procurement, security-review and investor questions without guessing. Facts marked “currently” reflect the deployed codebase on the date above.

CONTENTS

What this guide covers

A single source of truth for the product, the engineering, and the commercial model.

01 Executive summary
02 The problem we solve
03 Who we sell to
04 Product surfaces
05 Architecture
06 Request lifecycle
07 The 16 detection engines
08 Paid modules and pricing
09 Console: page by page
10 Free scanner and Vibe-Code
11 Client lifecycle
12 Payments, settlement and refunds
13 Data, identity and tenancy
14 Security model
15 Operations and testing
16 Known limitations and fixes
17 Client conversation guide
18 API and configuration appendix

> THE ONE-SENTENCE ANSWER
>
> Intellirity screens prompts, AI-generated code and agent actions before they reach a model or perform an action, then records evidence that the decision happened — sold as individual security modules rather than a seat-based platform.

How to use this document

- **Sales conversation:** read sections 2, 3, 8 and 17.
- **Technical evaluation:** read sections 5, 6, 7, 13 and 14.
- **Security review:** read sections 13, 14, 15 and 16.
- **Billing or partnership query:** read sections 8, 11 and 12.
- **Investor / strategy conversation:** read sections 1, 2, 8, 16 and 17.

01 · EXECUTIVE SUMMARY

What Intellirity is

Intellirity is an AI security platform. It checks the text, code and actions that surround a language model or autonomous agent, decides whether they are safe, and records proof of the decision.

AI applications are different from ordinary web applications. They accept free-form language, retrieve content from the internet, call tools, spend money and make decisions with incomplete instructions. A traditional firewall can see a request; it cannot see that a retrieved web page is trying to instruct the model, that an agent has been quietly redirected to a new goal, or that a generated code block contains a live database password.

Intellirity adds a security layer around those behaviours. It runs sixteen deterministic detection engines — no "ask another model whether this looks bad" dependency for the core verdict — and exposes them through a marketing site, a free scanner and an authenticated console. The commercial product is split into nine buyable modules so a customer can pay for the coverage they actually run.

Screen

Prompts, retrieved content, outputs, code and agent actions are checked before they are trusted.

Decide

Each engine returns a risk score, findings, severity and an allow / flag / block-style verdict.

Control

Agent escrow, action policy, proof of intent and spend ceilings turn a verdict into an enforced action.

Prove

A hash-chained ledger, verification and signed certificates make the decision reconstructable later.

What is live today

- **Public site** with engine catalogue, pricing, glossary, blog, research, contact, security model, policies and sitemap.
- **Free security scanner** at `/free-scan`: website URL or pasted code, no account.
- **Console** with 16 workspace pages for scanning, verdicts, leakage, content, agents, policies, escrow, proof, models and audit.
- **16 detection engines**, all deterministic analysers, exposed through one module API.
- **9 paid modules** from ₹79 to ₹399/month, 20% off when all nine are taken.
- **Clerk authentication**, per-user and per-organisation tenancy.
- **PayU hosted checkout** with signed hashes, callback verification, idempotent grants and a recovery path.
- **Postgres persistence** for orders, entitlements, scans, threats, workspace models, policies, ledger records, escrow and contacts.
- **Python SDK, JavaScript SDK, MCP server, Chrome extension and VPC sidecar** as integration surfaces.

> WHAT IS NOT FINISHED
>
> The autonomous "run exploits in a sandbox and prove them" part of Vibe-Code is not deployed because it requires a Docker worker and an LLM budget. PayU settlement is configured inside the PayU merchant dashboard, not in this codebase. Threat history and model/policy configuration are now durable in Postgres. Section 16 lists every remaining gap honestly.

02 · THE PROBLEM

The problem we solve

AI systems fail in ways that ordinary application security does not model. Intellirity was built around those failure modes, not around a generic “AI firewall” label.

2.1 The core risk: text becomes authority

In a normal application, data and instructions are structurally separate. In a language model, both arrive as text. A retrieved web page, PDF, email, code comment or tool result can contain instructions the model follows as if they came from the application owner. This is **indirect prompt injection**. A user can also try to override the system prompt directly — that is **direct prompt injection** or a **jailbreak**.

The consequence is not just an embarrassing answer. If the model has tools, the consequence is a database query, an email, a payment, a file write or a cloud API call.

2.2 The six practical problems customers bring us

Problem	What goes wrong	What Intellirity does
Prompt injection and jailbreaks	Users or retrieved content override policy, leak the system prompt, or unlock prohibited behaviour.	Layered pattern, encoding, persona and exfiltration detection before the model is called; input and output direction supported.
Unsafe AI-generated code	“Vibe-coded” features ship with SQL injection, XSS, exposed keys, SSRF or unsafe dependencies.	Static code, URL, repository, dependency, secrets, threat-model and SARIF review.
Data leaving through AI	PII, credentials, source code or regulated data appears in prompts, outputs or tool calls.	DLP with PII, secret, code-leak and custom-pattern detection plus data-flow destination tracing.
Agent abuse	An agent is redirected, misuses a tool, escalates privilege, poisons memory or calls an unauthorised MCP server.	Action policy, behavioural drift, MCP governance, workflow anomaly and spend protection.
Runaway cost	A loop, retry storm or abusive client burns API budget faster than a human can react.	Token, cost, call-count and repeating-pattern ceilings in the Denial-of-Wallet engine.
No evidence	After an incident nobody can reconstruct what the model saw, which control ran, who approved it, or whether logs were altered.	Hash-chained ledger, signed proof-of-intent certificates and escrow state transitions.

2.3 Why a WAF is not enough

A WAF understands HTTP methods, paths, headers and known payload signatures. It does not understand that a paragraph inside a support ticket is attempting to change an agent’s objective, or that a model output is about to be rendered as HTML. Intellirity sits at the semantic layer: before the model consumes text, before a generated artefact is trusted, and before an agent action is released.

2.4 The design principles behind the product

- **Deterministic first.** The detection engines are regex, statistical and state-machine analysers. They do not depend on an external LLM to produce a verdict.
- **Fail closed.** If an engine errors in the screening path, the request is denied rather than forwarded unchecked.
- **Evidence by default.** A verdict is useful only if it can be reconstructed. The ledger is append-only and hash-linked.
- **Per-module commerce.** A customer should not have to buy the entire platform to solve one problem.
- **Tenant isolation.** A user or Clerk organisation owns its own workspace, billing, models, policies and escrow.

03 · BUYERS

Who we sell to

The product is horizontal: any team putting a language model, AI-generated code or an autonomous agent into production has the same control problem.

Buyer	What they are building	Why they buy
AI product teams	Chatbots, copilots, summarisers, RAG applications, customer support automation.	Prompt injection, jailbreak and data-leak controls without building a detection team.
Platform / security teams	Internal AI platform used by many product teams.	One policy and evidence layer across models, agents and vendors.
Engineering teams using AI code generation	Fast-moving products written with coding assistants.	A free URL/code scan now, repository review and SARIF when the team is ready to formalise it.
Agent / automation builders	Workflows that call tools, spend money or act on customer records.	Action policy, MCP governance, escrow, human proof and spend limits.
Regulated or procurement-heavy organisations	Finance, healthcare, legal, public sector or enterprise vendors.	Evidence, tenant isolation, a sovereign sidecar option and an honest statement of certifications not yet held.
Agencies and consultancies	They build AI systems for clients and need a repeatable security check.	Free scanner for evaluation, per-module subscriptions for client work, repository review and reports.

Who it is not for (yet)

- Teams that only need a content filter and nothing else — the free engine may be enough; do not oversell.
- Buyers who require SOC 2 Type II or ISO 27001 today — we do not hold those certifications and do not claim them.
- Buyers who expect an autonomous penetration test that actively exploits a live target — that deep worker is not deployed.
- Buyers who need us to hold their money or act as a payment processor — PayU processes the payment; settlement is a PayU merchant-account function.

> POSITIONING LINE
>
> "We are the control and evidence layer around AI behaviour: screen the input, constrain the action, prove the decision. You buy only the controls you run."

04 · PRODUCT SURFACES

Every surface a client can touch

Intellirity is not just a website and a dashboard. There are seven usable surfaces, all backed by the same module registry.

Surface	Where it is	What it does
Marketing site	`intellirity.com`	Explains the problem, the 16 engines, pricing, policies and research; carries the SEO and answer-engine content.
Free scanner	`/free-scan`	No-account URL or code security scan. Real detector registry, results in the browser.
Console	`/console`	Authenticated workspace with 16 pages covering scans, verdicts, modules, policies, models, escrow, proof and audit.
Module API	`/backend/modules/{engine}/scan`	One authenticated endpoint per engine. Returns structured findings and a risk score.
Python SDK	`sdk/intellirity`	`from intellirity import Shield` — modules, trust machinery and proxy helpers.
JavaScript SDK	`sdk/js`	`import { Shield } from "@intellirity/shield"` plus `proxyChat`.
MCP server + extension	`mcp-server/`, `extension/`	15 MCP tools for Claude Desktop / MCP Inspector; Chrome MV3 popup that screens a pasted prompt.
VPC sidecar	`docker-compose.sidecar.yml`	Runs the engine beside the customer workload so prompts never leave their network.

Public pages

The indexable site includes: home, `/engines`, `/pricing`, `/free-scan`, `/glossary`, `/about`, `/blog` (10 articles), `/research`, `/careers`, `/contact`, `/refund-policy`, `/delivery`, `/privacy`, `/terms` and `/security`. The console is deliberately `noindex` and excluded from the sitemap.

05 · ARCHITECTURE

How the software is built

A Next.js frontend and a FastAPI engine, deployed as two services on Vercel with Postgres as the durable authority.

Visitor / client
Browser, SDK, MCP client or sidecar.

→

Next.js app
Marketing, free scanner, console UI, Clerk session, `/backend` proxy.

→

FastAPI engine
16 modules, risk scoring, auth, tenancy, rate limits.

→

Postgres
Orders, entitlements, scans, ledger, escrow, contacts.

Layer	Technology	Responsibility
Frontend	Next.js App Router, React, TypeScript, Tailwind-style tokens, Framer Motion, local Sora / Manrope / JetBrains Mono fonts.	Marketing pages, free scanner, console, SEO, structured data, Clerk UI, consent-gated analytics.
Backend	Python, FastAPI, Pydantic v2, SQLAlchemy async, Uvicorn; Mangum entrypoint for Vercel.	All 16 detection engines, auth, tenancy, billing, persistence, jobs, rate limits, health.
Database	PostgreSQL (Neon-compatible), asyncpg.	Durable records: users, organisations, scans, threats, ledger, escrow, orders, entitlements, contacts.
Identity	Clerk (RS256 JWKS / HS256 fallback), API keys hashed with SHA-256.	Human sessions and machine identities; org context decides tenant ownership.
Payments	PayU India hosted checkout, SHA-512 request/reverse hashes.	Checkout, callback, optional webhook, server-to-server payment recovery.
Email / abuse	Resend-style notification config, Cloudflare Turnstile, per-IP limits.	Contact submissions, bot protection, abuse ceilings.
Optional LLM	OpenAI-compatible upstream (opencode/zen, Groq, etc.).	Used only for reasoning/proxy features, never for the core detection verdict.

Deployment shape

- **Managed:** frontend + API on Vercel, Postgres hosted, Clerk and PayU as SaaS dependencies.
- **Sovereign:** the same engine as a sidecar container inside the customer's VPC; prompts stay inside the network.
- **Local/dev:** `uvicorn` on port 8000, Next on port 3000, optional Docker Compose with Postgres.

Important engineering detail: the browser never talks to FastAPI directly in production. Vercel rewrites `/backend/*` to the API service, so cookies, CSP and same-origin behaviour stay consistent.

06 · REQUEST LIFECYCLE

What happens when a scan runs

One request passes through identity, tenancy, plan enforcement, one or more engines, scoring, logging and evidence. This is the path to explain in any technical review.

1. **Identity.** The request carries either a Clerk bearer token (human), an `X-API-Key` (machine) or, in local development only, an open-mode identity. An invalid key is always rejected.
2. **Tenant resolution.** An active Clerk organisation becomes `org:<id>`; without one the user owns `user:<id>`; a machine key owns its own service tenant.
3. **Entitlement hydration.** The API loads that tenant's paid modules from Postgres when the instance cache is empty or older than 120 seconds. This is what makes a purchase work across serverless instances.
4. **Plan enforcement.** Each requested engine maps to a product. If the tenant does not own the product, the engine is denied. Free engines remain available.
5. **Input bounds.** Text fields are capped, code scans have a size limit, URL scans resolve DNS and refuse private, loopback, link-local or reserved addresses.
6. **Engine execution.** The engine runs deterministic rules, state machines or statistical checks. A multi-module sweep runs independent engines concurrently.
7. **Scoring.** Findings are weighted by severity; the module returns a risk score, findings, severity and verdict. Bands differ by engine but generally map to allow / flag / block.
8. **Response shaping.** The UI receives structured JSON. API responses are `private, no-store` with an authorization-aware `Vary` header.
9. **Evidence.** A high or medium risk event can create a threat record. Ledger writes are chained and persisted. Escrow transitions are validated and persisted.
10. **Activity.** A completed scan writes a small activity row (owner, type, risk, time) used by the real charts. The raw prompt is not written there.

What the client sees

The scanner returns a terminal-style report with a verdict, risk, per-engine findings and locked-module notices. The console overview shows the workspace's active modules, real scan volume, open threats and KPI strip. Free scanner users see only the relevant findings and fixes.

> FAILURE BEHAVIOUR
>
> If an engine raises, the module API returns an error instead of a fabricated verdict. In the screening-proxy path, a failure is a deny, not a pass. The UI surfaces a clear timeout or error rather than leaving a progress bar running forever.

07 · ENGINES

The 16 detection engines

Every engine is a real analyser registered in one feature registry. None of the core verdicts depends on a third-party language model. The tiers below are catalogue tiers; commercial packaging is in section 8.

#	Engine	Tier	Core job
01	**Jailbreak & Injection Protection**	free	Direct override, persona jailbreak, encoding/obfuscation and prompt-leak / exfiltration patterns on input and output.
02	**Advanced Threat Intelligence**	enterprise	Broader adversarial pattern families, behavioural drift, obfuscation, MCP abuse and multi-category correlation.
03	**Vibe Code Security**	professional	Static review of code, URLs, repositories and dependencies with findings, fixes, threat model and SARIF.
04	**AI Model Supply Chain Security**	enterprise	Model provenance, expected hash, unsafe pickle formats and known vulnerability checks before deployment.
05	**Data Loss Prevention (DLP)**	professional	PII, credentials, regulated data, source-code leaks and customer-defined patterns in model input/output.
06	**Data Flow Tracker**	professional	Every pipeline destination, trusted/blocked domains, sensitive-field access, destructive tools and external callbacks.
07	**Content Moderation**	professional	Seven content categories with layered severity scoring: spam, sexual, minor safety, profanity, violence, self-harm and hate/harassment.
08	**AI Action Policy Enforcer**	professional	Goal hijacking, tool misuse, privilege escalation, exfiltration, memory poisoning and denial-of-wallet at the action layer.
09	**Behavioural Analysis Engine**	professional	Session-history drift, anomalous action sequences and behavioural policy violations over time.
10	**MCP Security & Tool Governance**	enterprise	MCP server and tool-call review: poisoning, confused-deputy paths and unauthorised tool usage.
11	**Workflow Anomaly Detector**	professional	Hidden workflows, repeating loops, runaway spend, iteration/time limits and cascading side effects.
12	**Denial-of-Wallet Protector**	professional	Token, cost and call ceilings; repeating API patterns; resource-exhaustion and spend-abuse alerts.
13	**Autonomous Escrow**	premium	State-machine hold on agent funds until a trusted oracle releases or rejects; one decision only.
14	**Black Box Ledger**	premium	Append-only, hash-chained audit records with Merkle root, verification and query.
15	**Verifiable Proof of Intent**	premium	HMAC-signed certificate binding a human, an instruction hash, a scope, an expiry and a nonce.
16	**Threat Intelligence Feed**	enterprise	Catalogue of known AI attack patterns with type, severity and indicators for client consumption.

7.1 Jailbreak & Injection Protection

The free entry point and the most used engine. It combines seventeen pattern families and 113 patterns covering direct instruction override, “ignore previous instructions”, roleplay/persona jailbreaks, safety-disabling language, encoded payloads (base64 and similar), prompt-extraction attempts and exfiltration framing. It supports an input or output direction, returns a sanitized-text field, a recommendation and the list of modules triggered.

7.2 Advanced Threat Intelligence

A broader adversarial layer for teams that need more than the free engine. It maintains a threat catalogue and pattern families across prompt injection, jailbreak, obfuscation, data exfiltration, agent abuse, privilege escalation, MCP abuse, model supply chain, denial of service and behavioural drift. It correlates multiple signals rather than returning one regex hit.

7.3 Vibe Code Security

Two products share the name: a free public scanner for a URL or pasted code, and the paid repository review inside the console. The module looks for SQL injection, dynamic SQL, XSS sinks, command execution, SSRF, path traversal, insecure deserialization, weak crypto, hardcoded secrets, unsafe random generation, ReDoS and more. The repository path adds per-file line numbers, CWE, confidence, fix effort, a derived threat model, coverage, an ordered remediation plan and a SARIF 2.1.0 report. It does not execute the code.

7.4 AI Model Supply Chain Security

Reviews the model source and format before deployment. Unverified/community/unofficial sources are flagged; an expected SHA-256 can be compared; pickle formats are treated as code-execution risk; caller-supplied known vulnerabilities are included. It returns a block/flag/allow verdict and a deployment recommendation.

7.5 Data Loss Prevention

Detects email, US SSN, phone, credit-card-shaped numbers, IP addresses and date-of-birth context; cloud keys, GitHub tokens, Stripe keys, Google API keys, Slack tokens, JWTs, private-key blocks and password assignments; SQL/XSS/code-execution output; and customer-supplied regex patterns. The action escalates from `log` to `redact` to `block`.

7.6 Data Flow Tracker

Maps each pipeline stage and destination. It flags destinations on a blocked list or outside a trusted list, sensitive fields accessed by a sensitive/restricted classification, destructive tool calls, external callback URLs and excessive reasoning-chain length.

7.7 Content Moderation

Seven categories with independent pattern families and a weighted score. The engine deliberately allows mild profanity and keeps the severe terms narrow to reduce false positives. Minor-safety sexual framing is treated as critical.

7.8 AI Action Policy Enforcer

Evaluates an agent action, action type, tool calls, session context and policy object. It returns `allowed`, violations, abuse categories, risk score, critical count and a recommendation. This is the policy layer behind the Policies console page.

7.9 Behavioural Analysis Engine

Compares the current action against session history. It reports actions analysed, anomalies, behavioural drift, risk level and a recommendation. It is designed for traces, not one-off prompts.

7.10 MCP Security & Tool Governance

Checks MCP servers and tool calls for poisoning, confused-deputy patterns and tools outside the allowed set. It returns servers checked, tool calls checked, findings, risk and status.

7.11 Workflow Anomaly Detector

Ingests a workflow's actions with cost and duration. It detects repeated action sequences, excessive iterations, spend above a configured ceiling, timeouts and cascading side effects. The console turns a comma-separated action list into that payload.

7.12 Denial-of-Wallet Protector

Detects API loops, token and cost ceilings, unusual call counts and resource exhaustion. It returns alerts, total tokens, total cost, call count, risk and an action, and is wired to the Wallet page.

7.13 Autonomous Escrow

A real state machine: `held → released` or `held → rejected`, exactly once. It rejects unknown IDs, invalid amounts and replays. Holds are persisted in Postgres and restored across instances.

7.14 Black Box Ledger

Append-only records sealed with a hash that commits to the previous record; a Merkle root summarises the set. Verification walks the chain and reports tampering. In production the database is the authority: appends are serialized against the database tail so multiple processes cannot fork the chain.

7.15 Verifiable Proof of Intent

Issues an HMAC-SHA256 certificate containing the human ID, instruction hash, action scope, timestamp, expiry and nonce. Verification rejects a forged signature, an expired certificate and an action outside the signed scope. This is proof that a human authorised a class of action, not proof that the model behaved.

7.16 Threat Intelligence Feed

Returns a catalogue of known AI attack patterns with type, severity and indicators. **Currently this is a curated static catalogue of eight patterns, not a live external intelligence subscription.** It is useful as a reference and UI feed; it should not be sold as a continuously updated commercial threat feed without adding that data pipeline.

08 · COMMERCIAL MODEL

Modules, pricing and what a client buys

The commercial unit is a module for 30 days. There are no seats, no platform fee and no automatic renewal.

Product	Price	Engines included	Console destination
Injection Shield	₹99 / 30 days	Jailbreak & Injection Protection + Advanced Threat Intelligence	Scanner
Vibe-Code Security	₹99 / 30 days	Vibe Code Security + Model Supply Chain Security	Free scanner; repository review in console
Leakage Guard	₹99 / 30 days	Data Loss Prevention + Data Flow Tracker	Leakage, Data flow
Content Guard	₹79 / 30 days	Content Moderation	Content guard
Agent Guard	₹99 / 30 days	Action Policy, Behavioural Analysis, MCP Security, Workflow Anomaly	Agent behaviour, Policies, Anomalies, Models
Wallet Guard	₹79 / 30 days	Denial-of-Wallet Protector	Wallet guard
Agent Escrow	₹399 / 30 days	Autonomous Escrow	Agent escrow
Evidence Ledger	₹299 / 30 days	Black Box Ledger	Audit log
Proof of Intent	₹299 / 30 days	Verifiable Proof of Intent	Proof of intent

Commercial rule	Detail
Bundle	All nine modules: 20% off. Catalogue subtotal ₹1,551; bundle total ₹1,241 for 30 days.
Term	30 days per payment. Access continues to the end of the paid period.
Renewal	Manual. Early renewal extends from the current expiry; late renewal starts from now. Nothing auto-charges.
Free tier	Jailbreak engine, threat feed, free Vibe-Code URL/code scanner, console overview / scanner / verdicts / intel. A working free engine, not a time-limited trial of a different product.
Currency	Indian rupees (INR), processed by PayU India.
Vibe-Code nuance	The public URL/code scanner is free and requires no account. The paid product adds the repository review workspace. Be precise in sales: the free scanner is real; the paid module is the deeper repository workflow.

DO NOT SAY

“All modules are free.” Only the free tier and the public Vibe-Code scanner are free. “Unlimited scans” means good-faith scans under fair-use limits, not unlimited third-party targets or resale.

09 · CONSOLE

The console, page by page

Sixteen pages, grouped into Operate, Defend and Govern. Every page calls a real backend module or a real workspace API; the pages are not static mockups.

Page	Group	Plan	What the user does
Overview	Operate	Free	Security score, threats blocked/active, total scans, real scan-volume chart, open threat queue, active module strip and free tools.
Scanner	Operate	Page free; engines gated	Run profiles: full barrage, injection, exfiltration, content, advanced, wallet. Target an endpoint, watch the live report, queue a background sweep, keep session history.
Verdicts	Operate	Free	Severity-filtered verdict stream, search, detail drawer, resolve/acknowledge and paging.
Wallet guard	Operate	Wallet Guard	Enter session tokens, calls and cost; run the Denial-of-Wallet engine; review alerts and action.
Models & agents	Operate	Agent Guard	Register model endpoints, pause/resume, run supply-chain verification, record scan results.
Leakage guard	Defend	Leakage Guard	Choose PII / credential / code detectors, scan text, see matches and the log/redact/block action.
Anomalies	Defend	Agent Guard	Turn an action list into a workflow payload; detect loops, spend, iteration and side-effect anomalies.
Data flow	Defend	Leakage Guard	Define pipeline destinations, trusted and blocked domains; see destinations, flags and risk.
Content guard	Defend	Content Guard	Screen input or output across the seven moderation categories; see verdict, findings and score.
Agent behaviour	Defend	Agent Guard	Behavioural drift plus MCP server/tool governance in one workspace.
Vibe-Code Security	Defend	Free scanner; repo paid	Free URL/code scan at the top; repository file/folder review for owners, with threat model, SARIF and remediation plan.
Threat intel	Defend	Free feed; advanced gated	Browse the attack-pattern catalogue and run the advanced threat-intelligence engine on a prompt or pattern.
Proof of intent	Govern	Proof of Intent	Issue a signed human-intent certificate; verify signature, expiry and action scope.
Policies	Govern	Agent Guard	Enable/disable guardrail packs, inspect YAML, test an action against the policy, promote a pack through approval.
Agent escrow	Govern	Agent Escrow	Create, verify/release, reject and inspect agent-fund holds with exactly-once transitions.
Audit log	Govern	Evidence Ledger	Seal a decision, search/filter entries, inspect hashes, verify the chain and export.

Free versus paid in the console

Overview, Scanner, Verdicts and Threat Intel are reachable without a paid module. A paid engine still requires its product. Pages such as Leakage, Content, Wallet, Models, Policies, Escrow, Proof and Audit are gated by their product. The code page is not gated at the route level because its free scanner is public; the repository section only renders for a workspace that owns Vibe-Code.

10 · FREE SCANNER

Vibe-Code Security, free and paid

This is the lowest-friction product surface and one of the most important differentiators. A visitor can get a real security result without signing in, paying or reading documentation.

10.1 The free public scanner

URL: **https://www.intellirity.com/free-scan**. Two modes:

- **Website URL:** resolves the host through the SSRF guard; refuses private, loopback, link-local and reserved addresses; does not follow redirects blindly; reads up to 300 KB of HTML; checks transport, HSTS, CSP, X-Frame-Options, X-Content-Type-Options, X-Powered-By, CORS, cookie flags, mixed content, external scripts without SRI, inline-script code patterns and page-source secrets.
- **Paste code:** runs the same Vibe-Code detector registry on up to 50,000 characters. Findings include severity, CWE, line and a concrete fix.

Limits are deliberate: a scan is not an exploit, does not authenticate, does not crawl the whole site and does not brute-force endpoints. The UI states this. The endpoint is rate-limited and records only activity metadata (type, risk, time), not the scanned URL or code.

10.2 The paid repository review

Inside `/console/code`, a workspace that owns Vibe-Code can choose a folder or files. The browser reads them locally and sends a bounded JSON snapshot:

Limit	Value
Maximum files	300
Maximum single file	200,000 characters
Maximum total snapshot	1,500,000 characters
Skipped files	Binaries, assets, duplicates, oversized files and exhausted budget are listed in coverage

The review returns: per-file findings with line, rule ID, severity, CWE, confidence and fix effort; an ordered remediation plan; a threat model derived from routes, auth, uploads, outbound calls, shell execution, data stores and literal secrets; coverage with skipped-file reasons; and a SARIF 2.1.0 download for CI.

Repeated findings are grouped per rule and file, with a true occurrence count, and the report is capped at 300 groups. This prevents a repeated line from creating a 15 MB response.

10.3 Dependency and manifest review

Package manifests are parsed for risky and unpinned dependencies. The current risky-package list includes vm2, node-serialize, event-stream, serialize-javascript, PyCrypto, request, node-sass, moment and left-pad. This is a review prompt, not a live CVE database; versions are reported as written and no lockfile resolution is performed.

10.4 What it does not do

- It does not launch a Docker sandbox or actively exploit a target.
- It does not authenticate to the customer's application or crawl behind a login.

- It does not guarantee absence of vulnerabilities after a clean scan.
- It does not replace a manual penetration test or a compliance audit.

10.5 The autonomous half

The reference project behind the Vibe-Code concept includes an autonomous agent that plans tests, uses tools, runs a sandbox and writes exploit evidence. That requires a Docker-capable worker, outbound internet and an LLM API key. It cannot run inside a request-scoped Vercel function. Until that worker is deployed, do not sell “autonomous exploit validation”. The static scanner, repository review, threat model, coverage and SARIF are live.

11 · CLIENTS

How we deal with clients, end to end

The motion is self-serve. A client can evaluate, pay and activate without a sales call; higher-touch work is reserved for procurement, sovereign deployment and security review.

1 · Discover
Marketing site, blog, engine catalogue, free scanner, direct link.

→

2 · Evaluate
Free URL/code scan, free injection engine, threat feed.

→

3 · Sign in
Clerk account, personal or organisation workspace.

4 · Choose coverage
Toggle the modules; price is live and the bundle discount is applied.

→

5 · Pay
PayU hosted checkout; no card stored by Intellirity.

→

6 · Activate
Verified callback or recovery grants the modules for 30 days.

7 · Operate
Scanner, verdicts, leakage, agents, policies, models, audit.

→

8 · Prove
Ledger, proof of intent, escrow.

→

9 · Renew or stop
Manual renewal; no auto-charge; access runs to expiry.

11.1 First contact

Most clients arrive through search or a direct link. The free scanner is the shortest proof: they paste a URL or code and see a real result. The free injection engine on the landing page or scanner is the second proof. No sign-in is needed for either.

11.2 Sign-in and workspace

Clerk handles sign-in. A user without an organisation owns a private tenant. A user with an active Clerk organisation shares that organisation's billing, models, policies and escrow. The console requires a session; machine clients use API keys. This distinction matters in a sales call: a team that wants shared billing must use a Clerk organisation, not just share a login.

11.3 Purchase

The pricing builder shows the live catalogue, selected modules, subtotal, bundle discount and total. Checkout collects name, email and phone for PayU, creates an order, persists it and returns a signed form. PayU hosts the payment page. On return, the server verifies the reverse hash; the modules are granted only on success. A recovery endpoint can reconcile a captured payment if the browser callback was lost.

11.4 Onboarding after payment

- Refresh the console; the "your modules" strip shows the active modules and renewal date.
- Open the matching console page; the route unlocks when the entitlement is hydrated.
- Register model endpoints under Models & agents if using Agent Guard.
- Inspect and enable policy packs under Policies.
- Seal an audit record under Audit log if Evidence Ledger is purchased.

11.5 Support model

Channel	Address	Use
Sales and billing	security@intellirity.com	Modules, invoices, GST details, payment that needs attention.
Security reports	security@intellirity.com	Vulnerability reports, novel bypasses, coordinated disclosure.
Privacy	privacy@intellirity.com	Data-subject requests, retention, subprocessors.
Legal	legal@intellirity.com	Contracts, terms, DPAs, notices.

Security reports are triaged first. The stated policy is acknowledgement within 24 hours, a fix or mitigation within 30 days, and public credit unless the researcher prefers anonymity. Billing messages are read by the founder; include the workspace email or order ID.

11.6 Delivery, cancellation and refund

- **Delivery:** digital; modules activate in the workspace normally within seconds. If a confirmed payment has not activated access within 30 minutes, email with the order ID.
- **Cancellation:** no auto-renew. Cancel by not renewing. Access continues to the end of the paid 30 days.
- **Refunds:** duplicate/accidental charge, a successful payment that cannot be activated, an unauthorised transaction reported promptly, or a material outage proportional to the paid period. Not automatic for change of mind after use or an elapsed period.
- **Process:** email the order ID, workspace email and reason. Acknowledgement within two working days; decision within seven; approved refunds go back through PayU, typically 5–10 working days after approval.

11.7 Enterprise and sovereign

The engine can run as a sidecar inside the customer's VPC. That is the answer for a client who cannot send prompts to a hosted API. It is not a different detection engine; it is the same module registry deployed beside the workload. Pricing and support for that deployment are a separate commercial conversation, not the self-serve ₹79–₹399 list.

Payments, settlement and refunds

The checkout is a PayU hosted redirect. The code verifies payment; it does not move money. Settlement is a merchant-bank function inside PayU.

12.1 Checkout flow

1. The signed-in buyer selects modules and enters name, email and a valid phone number.
2. The server prices the order from the catalogue and stores it in Postgres with status `pending-payment`.
3. The server builds a PayU form with a SHA-512 request hash: key, txnid, amount, productinfo, firstname, email, udf1–udf5 and salt.
4. The browser POSTs the form to PayU's hosted checkout.
5. PayU posts the result to `/backend/billing/payu/callback`. The server verifies the reverse hash.
6. On success, the order becomes `active` and entitlements are written to Postgres. Replays are idempotent; an existing order is not granted twice.
7. If the callback is lost, the pricing page calls `/payu/sync`. The server asks PayU directly, checks the captured amount equals the order total, and grants the modules.

12.2 What the client must configure in PayU

Setting	Value
Success URL	`https://www.intellirity.com/backend/billing/payu/callback`
Failure URL	`https://www.intellirity.com/backend/billing/payu/callback`
Webhook URL (optional)	`https://www.intellirity.com/backend/billing/payu/webhook`
Website / product URL	`https://www.intellirity.com/pricing`
Mode	Live, with the live merchant key/salt in the backend environment.

The webhook is optional for this redirect flow. Opening the webhook URL in a browser returns a normal explanation page; it only accepts a signed server-to-server POST. The "enhance your customers' checkout experience" prompt in the PayU dashboard is checkout customization, not settlement. It can be skipped.

12.3 Settlement: why money may not reach the bank

A PayU dashboard transaction can be successful while the money is still held or routed to a different bank account. The application cannot change this. The merchant account controls it.

Check	What to verify
Merchant account	The dashboard MID and merchant key must match the key used by the backend. A different PayU account has a different bank account.
Live vs test	Test transactions show amounts but never settle. Confirm the transaction is in the live dashboard.
Bank details	A verified settlement bank account must exist on the MID: correct account holder, account number and IFSC, with penny-drop/verification complete.

Check	What to verify
KYC / activation	An account can accept transactions while settlement is on hold pending KYC, website verification or activation.
Settlement cycle	Usually T+1/T+2 business days. Weekends and bank holidays shift the date.
Minimum payout	Some merchant accounts have a minimum settlement amount. A single small payment can remain below the threshold.
UTR	If the transaction shows a UTR, PayU released the money; give the UTR to the bank. No UTR beyond the cycle means a PayU support case.
Reserve / hold	Rolling reserve, chargeback hold or risk review can delay settlement.

> SALES-SAFE ANSWER
>
> "PayU confirms the payment to us and we unlock the modules immediately. Settlement to your bank is handled by your PayU merchant account. If the dashboard shows a successful transaction but the bank has not received it, send the txnid, mihpayid and UTR (if any) to PayU support with your MID."

12.4 Refund mechanics

Refunds are issued through PayU against the original transaction. Intellirity does not store card numbers or UPI credentials. A refund request needs the order ID, workspace email and reason; approval goes back to the original payment method.

Data model, identity and tenancy

The tenant is the unit of isolation. A user or organisation owns its records; a machine key owns its own service workspace.

Table	What it holds	Durability
users	Human user records.	Postgres
organizations	Team tenant records.	Postgres / memory fallback
security_scans	Completed scan metadata and activity used by real charts.	Postgres
threat_detections	Schema exists for durable detections; current console threat store is in-process.	Table exists; store still memory-first
ledger_records	Hash-chained audit records, sequence and Merkle hash.	Postgres is authoritative
escrow_holds	Agent fund holds and state.	Postgres, restored across instances
orders	Checkout, modules, amount, status, provider txnid, paid time.	Postgres
entitlements	Owner, product, order and expiry.	Postgres
contact_submissions	Ticket, name, email, subject, message, notified flag.	Postgres before email

Identity paths

- **Human:** Clerk session JWT verified with RS256/JWKS (or HS256 fallback). Expired, forged or unsigned tokens are rejected.
- **Machine:** `X-API-Key`, matched by SHA-256 hash; no raw key stored.
- **Open mode:** only when no auth is configured and the environment is not production. Production without auth refuses to serve protected routes.

Tenant rules

An active Clerk organisation owns the workspace. Without an organisation, the user owns a private tenant. A machine key owns its own service tenant and is never written to the user table. Cross-tenant reads are 404, not “forbidden” with an existence leak.

Persistence caveat

Orders, entitlements, scan activity, threats, model endpoints, policy packs, ledger, escrow and contacts are durable in Postgres. The in-process store remains a fast cache/fallback, but a serverless cold start no longer erases the tenant’s threat history or workspace configuration. The audit ledger remains the permanent evidence record.

14 · SECURITY

The security model

The product is sold as security software, so its own controls have to be explainable. This is what the deployed system actually enforces.

Control	Implementation
Fail closed	A screening-engine error denies rather than forwards. An invalid API key is 401 even in open mode. A bearer token presented to a deployment that cannot validate it is 503, not anonymous access.
Tenant isolation	Owner is derived per request; engine responses are private/no-store with `Vary: Authorization`; cross-tenant object reads are 404.
Input bounds	Text/code size caps, file-count and total-size caps for repository review, URL length caps, bounded public scan payloads.
SSRF protection	URL scans resolve DNS, reject private/loopback/link-local/reserved/multicast addresses, and do not follow redirects blindly.
Evidence integrity	Append-only hash chain with Merkle root; database-authoritative append lock; verification command detects a broken chain.
Intent proof	HMAC-SHA256 certificate over human ID, instruction hash, action scope, timestamp, expiry and nonce; forged/expired/out-of-scope certificates are denied.
Escrow integrity	Exactly-once state machine; unknown IDs and replays fail; holds persisted and restored.
Web hardening	CSP, HSTS with preload, X-Frame-Options DENY, nosniff, strict referrer, COOP/CORP, permissions policy, no `X-Powered-By`, interactive API docs disabled in production.
Abuse control	Per-IP request budget, tighter per-endpoint limits for contact/reasoning/jobs/demo scans, Turnstile on contact when configured.
Privacy	Prompt payloads are not retained by default; contacts are written before email attempt; analytics is consent-gated; no training on customer traffic.
Secrets	No live keys in tracked files; `.env` is ignored; API keys stored hashed; PayU salt stays server-side.

> CERTIFICATION ANSWER
>
> Intellirity has **not** completed SOC 2 Type II or ISO/IEC 27001. We do not publish badges we cannot evidence. What we can show is the control implementation above, the policy pages, and the code-level tests.

15 · OPERATIONS

How the product is run and tested

The project is small enough to operate without a large platform team, but it has real operational controls.

15.1 Deployment

- Frontend and API on Vercel; Postgres hosted (Neon-compatible); Clerk and PayU as SaaS.
- Vercel rewrites `/backend/*` to the API service; the browser stays same-origin.
- Canonical host is `www.intellirity.com`; apex redirects. Sitemap, robots and canonicals use www.
- Cutover and rollback are Git-based; the latest tested commit is deployed through the repository.

15.2 Health and backup

- `GET /health` and `GET /backend/system/health` return 200 healthy / 503 when a production deployment cannot reach Postgres.
- Operational scripts: ledger verification, database backup, database maintenance, ledger repair dry-run, engine catalogue export.
- Backups are database-level; the policy states 35-day backup retention and 30-day account-data retention after closure.

15.3 Testing

- **Backend:** 104 tests pass, with database disabled in the test suite so tests cannot write to production.
- **Frontend:** TypeScript check and production Next.js build pass.
- **Security dependency audit:** `npm audit` 0 known vulnerabilities; `pip-audit` no known vulnerabilities at the time of writing.
- **SDK suites:** Python and JavaScript live tests exist against a running engine.
- **Site checks:** 94/94 live checks pass across public pages, canonicals, robots and sitemap.

15.4 What monitoring does not cover yet

There is no external uptime pager in this document, no formal incident-response on-call rotation, no WAF in front of the app, and no SIEM export. Those are operational purchases/processes to add as the customer base grows.

16 · GAPS

Known limitations and how to fix them

This section is deliberately blunt. A client will find these eventually; it is better to explain them correctly and price/position around them.

Gap	Impact	What is true today	Fix
PayU settlement	business-critical	The app verifies successful payments. The money is settled by PayU to the MID's bank account.	Verify MID/key match, live mode, bank verification, KYC, cycle, minimum payout, UTR; raise PayU support with txnid/mihpayid.
Advanced threat feed	low	The feed is a curated eight-pattern catalogue. Its public description now says "curated catalogue", not "live feed".	Add an ingestion pipeline with provenance and freshness dates if a live subscription is sold.
Autonomous exploit validation	scope	Static URL/code/repo review only. No sandbox, no exploit proof.	Deploy a Docker-capable worker with an LLM key and a bounded job queue.
General API rate limit	low	Costly routes (contact, reasoning, jobs, public scans) are now counted in Postgres across instances. The broad 60/min middleware still has an in-process fallback.	Move the broad counter to Redis/Upstash for a fully global limit.
CSP unsafe-inline	low	Script-src allows inline scripts for the current Next/Clerk setup.	Nonce/hash-based CSP; test third-party scripts and Clerk.
Email deliverability	medium	The published security/privacy/legal addresses are on a domain with no MX record in DNS checks. Contact-form outbound alerts are configured.	Add MX/SPF/DMARC and create the mailboxes, or route support through the contact form until then.
Business details	low	The contact page shows "Available on request" for address/phone and no GSTIN when the environment variables are unset.	Set `NEXT_PUBLIC_COMPANY_ADDRESS`, `PHONE`, `GSTIN` and redeploy.
Refund UI	low	Refunds are handled by email and the PayU dashboard; no in-app refund button.	Add an admin refund workflow if volume justifies it.
Autonomous URL crawling	scope	Free URL scan reads one public page and headers; it does not crawl a whole site or authenticate.	Add a bounded crawler worker if clients need full-site coverage.
Certifications	procurement	No SOC 2 Type II or ISO 27001.	Commission an audit when enterprise demand justifies it; do not imply certification before then.

MOST IMPORTANT REMAINING OPERATIONAL ITEMS

PayU settlement and bank verification are merchant-account tasks. Inbound support email needs MX/SPF/DMARC and real mailboxes. The autonomous exploit worker still needs a Docker host and an LLM key. The code-level durability and public-health issues are fixed.

How to answer the questions you will actually get

Short, accurate answers. Do not improvise beyond them.

Q1

“What exactly do you sell?”

Per-module AI security for 30 days. Nine modules from ₹79 to ₹399; free tier and free Vibe-Code scanner for evaluation. No seats, no platform fee, no auto-renew.

Q2

“Is this a WAF?”

No. A WAF understands HTTP. Intellirity understands intent: semantic injection, jailbreaks, agent actions, data flow, code and evidence.

Q3

“Does it call an LLM to decide?”

The core detection engines are deterministic regex/statistical/state-machine analysers. An LLM is used only for the optional reasoning/proxy features.

Q4

“Do you store our prompts?”

No by default. We process to return a verdict and store metadata/evidence according to the workspace retention policy. Scan activity stores type, risk and time, not payload.

Q5

“Can it stop an agent?”

It can evaluate the action, enforce policy, hold funds in escrow and require a signed human intent proof. It is a control layer, not a guarantee that the agent is correct.

Q6

“How fast is it?”

Most deterministic scans complete in well under a second; a URL scan adds a bounded page fetch up to eight seconds; repository review is a few seconds depending on size.

Q7

“What if an engine crashes?”

The screening path fails closed: deny rather than forward. The UI reports an error; it does not fabricate a clean verdict.

Q8

“Are you SOC 2 / ISO certified?”

Not yet. We do not claim certifications we do not hold. We can show the security model, controls, tests and policies.

Q9

“Can we self-host?”

Yes, as a sidecar inside your VPC. That is a separate deployment and commercial conversation.

Q10

“How do we pay?”

PayU hosted checkout in INR. Each payment covers 30 days. No card is stored by us; settlement is to the PayU merchant account.

Q11

“What is free?”

The injection engine, threat feed, free Vibe-Code URL/code scan and the free console pages. Paid modules unlock the deeper engines and repository workflow.

Q12

“What is missing?”

Autonomous exploit validation, durable threat/model/policy state, external intelligence ingestion, SOC 2/ISO and in-app refunds. Section 16 lists them.

The strongest pitch is not “we stop every attack.” It is: “we put a deterministic control and a reconstructable record around the places where AI text becomes an action — and you only pay for the controls you run.”

API and configuration reference

Enough to answer an integration question without opening the repository.

18.1 Core API groups (prefix `/api/v1`)

Group	Routes
System	status, health, summary, public stats/feed, free Vibe-Code scan, demo prompt scan, contact.
Modules	list modules, get module, `POST /{engine}/scan`.
Scans	history, code-security, create, run sweep, activity, get by id.
Threats	list/filter, get, resolve.
Billing	products, checkout, entitlements, orders, PayU callback/webhook/sync/urls, dev activate.
Workspace	model endpoints, model status, scan result, policy packs, enable, promote.
Identity / orgs	me, stats, organisations.
Jobs / realtime	queue a sweep/reasoning job, job status, ingest/feed.
Proxy / reasoning	screened chat proxy and optional reasoning endpoint.

18.2 Environment variables that matter in a client conversation

Variable	Purpose
`DATABASE_URL`	Postgres connection; blank means memory-only local mode.
`CLERK_SECRET_KEY` / `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY`	Human authentication and tenant context.
`INTELLIRITY_API_KEY_HASHES`	SHA-256 hashes of machine keys for SDK/MCP.
`PAYU_KEY_ID` / `PAYU_KEY_SECRET` / `PAYU_MODE`	PayU merchant credentials and live/test selection.
`PUBLIC_SITE_URL`	Fallback origin for payment return URLs.
`SECRET_KEY` / `VPI_SIGNING_KEY`	App signing and proof-of-intent HMAC key.
`ALLOWED_ORIGINS`	CORS allow-list; never * in production.
`BILLING_ENFORCED`	Turns module entitlements on.
`TURNSTILE_SECRET`	Contact-form bot protection.
`NEXT_PUBLIC_SITE_URL` / `NEXT_PUBLIC_COMPANY_*`	Canonical origin and legal business details.

18.3 Glossary for client calls

Term	Meaning in this product
Prompt injection	Text treated as data that the model follows as an instruction. Direct = user prompt; indirect = retrieved page/document/tool result.
Jailbreak	An attempt to override safety or policy behaviour, often through persona, roleplay, encoding or multi-turn framing.
MCP	Model Context Protocol — a standard way for an agent to discover and call tools/servers. Adds a supply-chain and confused-deputy surface.
Confused deputy	A trusted component is tricked into using its authority on behalf of an attacker.
Denial of wallet	An attack that burns API/token spend rather than taking the service down.
Proof of intent	A signed statement that a specific human authorised an instruction/action scope until an expiry.
Escrow	A hold on agent funds until an oracle verifies the deliverable; released or rejected exactly once.
Ledger	Append-only, hash-chained evidence of decisions; tampering breaks verification.
SARIF	Standard JSON format for static-analysis results, consumed by code-scanning/CI tools.
Fail closed	On error, deny rather than allow.

Document status: based on the deployed codebase and live site as of 7 November 2026. Product claims in marketing copy should be checked against this document before publication. Known gaps in section 16 are part of the record, not omissions.